实时通知与推送方案

> 文档版本: v1.2
>
> 更新日期: 2026-04-13
>
> 对应 **PRD**: v1.8 §2.3 智能预警引擎
>
> 对应 **data_model**: v1.5（alerts 表 + notifications 表 v1.8 新增）
>
> 对应 **API_CONTRACT**: v1.8 §8A（通知系统 API）

一、通知渠道矩阵

渠道	平台	Phase 1	实时性	说明
系统内消息中心	全平台	✅	准实时（轮询）	写入 `alerts` 表，前端定期拉取
邮件	全平台	✅	延迟（分钟级）	SMTP 发送，失败自动重试
FCM 推送	iOS / Android / HarmonyOS	✅	实时	FCM 统一推送通道（集成 APNs/FCM/华为推送）
应用内 **Badge**	macOS / Windows / Web	✅	准实时	未读数轮询，角标显示
企业微信 **Webhook**	—	❌ Phase 2	实时	预留接口
飞书 **Webhook**	—	❌ Phase 2	实时	预留接口

一-A、通知系统双表模型（v1.8 新增）

Phase 1 存在两个通知相关表，职责分工如下：

表	来源	职责	前端入口
`alerts`	定时任务自动生成	预警：到期预警、逾期预警、工单超时等业务规则触发	Admin: 预警中心 `/settings/alerts`
`notifications`	API 手动/业务事件写入	通知：审批结果、催收提醒、系统公告等操作型通知	Admin + Flutter: 通知中心 `/notifications`

`notification_type` 枚举值（对应 data_model v1.5 §九-A）：

值	含义	推送渠道
`contract_expiry`	合同到期提醒	系统内 + 邮件 + Push
`invoice_overdue`	账单逾期通知	系统内 + 邮件 + Push
`workorder_update`	工单状态变更	系统内 + Push
`approval_pending`	审批待处理	系统内 + 邮件 + Push
`approval_result`	审批结果通知	系统内 + Push
`dunning_reminder`	催收提醒	系统内 + 邮件
`system_announcement`	系统公告	系统内

统一未读数：前端使用 `GET /api/notifications/unread-count`（API_CONTRACT §8A.3）获取通知未读数，与原 `GET /api/alerts/unread` 预警未读数分别展示。

二、Phase 1 实现方案

2.1 整体流程

```
┌──────────────┐      ┌──────────────┐      ┌──────────────────┐
│ 定时任务 /   │      │              │      │ 系统内消息中心   │
│ 业务触发     │──────►│ NotifyService │──────►│ (alerts 表)      │
│              │      │              │      └──────────────────┘
└──────────────┘      │              │
                      │              │      ┌──────────────────┐
                      │              │──────►│ EmailService     │
                      │              │      │ (SMTP)           │
                      │              │      └──────────────────┘
                      │              │
                      │              │      ┌──────────────────┐
                      │              │──────►│ FCM Push         │
                      │              │      │ (iOS/Android)    │
                      └──────────────┘      └──────────────────┘
```

2.2 轮询策略（桌面端 + Web）

参数	值	说明
预警轮询端点	`GET /api/alerts/unread`	返回 `{ data: { count: N } }`
通知轮询端点	`GET /api/notifications/unread-count`（v1.8 新增）	返回 `{ data: { count: N } }`
预警轮询间隔	30 秒	在 `ui_constants.ts` 定义 `ALERT_POLL_INTERVAL`
通知轮询间隔	60 秒	在 `ui_constants.ts` 定义 `NOTIFICATION_POLL_INTERVAL`（v1.8 新增）
应用前台	正常轮询	30 秒一次
应用后台	暂停轮询	Flutter 通过 `WidgetsBindingObserver` 生命周期检测；Admin 通过 `visibilitychange` 事件检测
未读 > 0	Badge 显示	导航栏铃铛图标 + 数字角标

2.3 FCM 推送（移动端）

后端集成

```
/// backend/lib/services/push_service.dart
class PushService {
  final String _fcmServerKey; // 从环境变量 FCM_SERVER_KEY 读取

  /// 发送推送通知（通过 FCM 服务端 API）
  Future<void> sendPush({
    required String deviceToken,
    required String title,
    required String body,
    Map<String, String>? data,
  }) async {
    // 调用 FCM HTTP v1 API 发送推送
    // 支持 APNs(iOS) / FCM(Android) / 华为推送(HarmonyOS) 统一通道
  }
}
```

Flutter 端集成

```
// flutter_app/lib/core/services/push_service.dart
import 'package:firebase_messaging/firebase_messaging.dart';

class FlutterPushService {
  final FirebaseMessaging _messaging = FirebaseMessaging.instance;

  /// 获取 FCM device token，上报给后端
```

```
  Future<void> registerDeviceToken() async {
    final token = await _messaging.getToken();
    if (token != null) {
      await apiPost('/api/users/me/device-token', {'token': token});
    }
  }

  /// 前台消息处理
  void setupForegroundHandler() {
    FirebaseMessaging.onMessage.listen((RemoteMessage message) {
      // 更新本地未读数 + 显示 SnackBar
    });
  }

  /// 点击通知消息导航
  void setupNotificationTapHandler() {
    FirebaseMessaging.onMessageOpenedApp.listen((RemoteMessage message) {
      final alertId = message.data['alert_id'];
      if (alertId != null) {
        // 通过 go_router 导航到对应页面
        // context.push('/alerts/detail/$alertId');
      }
    });
  }
}
```

> 微信小程序：不支持 FCM，使用微信模板消息（订阅消息）替代，需单独申请模板。

设备令牌管理

```
-- 新增表：user_device_tokens
CREATE TABLE user_device_tokens (
    id           UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id      UUID         NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    device_token TEXT         NOT NULL,
    platform     VARCHAR(20)  NOT NULL, -- 'ios', 'android', 'harmony', 'wechat_mp'
    is_active    BOOLEAN      NOT NULL DEFAULT TRUE,
    created_at   TIMESTAMPTZ  NOT NULL DEFAULT NOW(),
    updated_at   TIMESTAMPTZ  NOT NULL DEFAULT NOW(),
    UNIQUE (user_id, device_token)
);
```

三、NotifyService 实现

```
/// backend/lib/services/notify_service.dart
class NotifyService {
  final AlertRepository _alertRepo;
  final NotificationRepository _notificationRepo; // v1.8 新增：通知表写入
  final EmailService _emailService;
  final FcmService? _fcmService; // 可选，未配置 FCM_SERVER_KEY 时为 null
```

```
final PushService? _pushService; // 可选，未配置UNI_PUSH_APP_ID 时为null

/// 发送通知（多渠道分发）
Future<void> notify({
  required String alertId,
  required String title,
  required String message,
  UUID? targetUserId,          // 指定用户；null 表示按角色广播
  List<String>? targetRoles,   // 广播角色列表（对应alerts.target_roles user_role[] 字段）
}) async {
  final recipients = targetUserId != null
      ? [targetUserId]
      : await _userRepo.findByRoles(targetRoles!);

  final notifiedVia = <String>[];

  // 1. 系统内消息（始终写入）
  await _alertRepo.create(alertId: alertId, ...);
  // v1.8: 同时写入notifications 表（操作型通知）
  await _notificationRepo.create(
    userId: targetUserId,
    type: notificationType,   // notification_type 枚举
    severity: severity,       // notification_severity 枚举
    title: title,
    content: message,
    resourceType: resourceType,
    resourceId: resourceId,
  );
  notifiedVia.add('in_app');

  // 2. 邮件
  for (final uid in recipients) {
    final user = await _userRepo.findById(uid);
    if (user.email != null) {
      try {
        await _emailService.send(to: user.email!, subject: title, body: message);
        notifiedVia.add('email');
      } catch (e) {
        // 邮件失败记录日志，不阻塞
        await _logEmailFailure(alertId, uid, e);
      }
    }
  }

  // 3. FCM 推送（仅移动端用户）
  if (_pushService != null) {
    final tokens = await _deviceTokenRepo.findActiveByUsers(recipients);
    for (final t in tokens) {
      try {
        await _pushService!.sendPush(
          clientId: t.deviceToken,
          title: title,
```

```
            body: message,
            data: {'alert_id': alertId},
          );
          if (!notifiedVia.contains('push')) notifiedVia.add('push');
        } catch (e) {
          await _logPushFailure(alertId, t.userId, e);
        }
      }
    }

    // 4. 更新 notified_via
    await _alertRepo.updateNotifiedVia(alertId, notifiedVia);
  }
}
```

四、预警接收角色映射

预警类型	目标角色	说明
`lease_expiry_90/60/30`	`leasing_specialist` , `operations_manager`	招商 + 运营经理
`payment_overdue_1/7/15`	`finance_staff` , `leasing_specialist`	财务 + 招商
`monthly_expiry_summary`	`operations_manager` , `super_admin` , `report_viewer`	管理层 + 只读观察者
`deposit_refund_reminder`	`finance_staff`	财务专员
`workorder_assigned`	`maintenance_staff`	维修技工（工单派单通知）
`workorder_overdue`	`maintenance_staff` , `operations_manager`	超期工单预警

五、防重复轰炸规则

规则	实现
同合同 + 同 alert_type + 同日	`UNIQUE(contract_id, alert_type, DATE(triggered_at))`
邮件发送失败重试	最多 3 次，间隔 30s / 60s / 120s
FCM 令牌失效	收到 `UNREGISTERED` 响应后标记 `is_active = false`

六、环境变量

变量	必填	说明
`SMTP_HOST`	Phase 1 可选	SMTP 服务器地址
`SMTP_PORT`	Phase 1 可选	SMTP 端口（默认 587）
`SMTP_USER`	Phase 1 可选	SMTP 用户名
`SMTP_PASSWORD`	Phase 1 可选	SMTP 密码
`SMTP_FROM`	Phase 1 可选	发件人地址
`FCM_SERVER_KEY`	Phase 1 可选	Firebase 服务端密钥（未配置则跳过 FCM 推送）

> **Phase 1 最小启动**：即使不配 SMTP/FCM，系统内消息中心 + 轮询轮询仍可正常工作，仅降级为"仅系统内通知"。